

5

HỌC MÁY  
TĂNG CƯỜNG CHO  
CÁC HỆ THỐNG MẠNG

# Continuous & Policy-Based RL với DDPG và PPO

Biên soạn: ThS. Phan Trung Phát  
Liên hệ: [phatpt@uit.edu.vn](mailto:phatpt@uit.edu.vn)

Tái bản lần 1 -- Tháng 03/2026

Lưu hành nội bộ

## A. MỤC TIÊU

- Hiểu được sự khác biệt giữa các phương pháp học tăng cường sâu dựa trên policy và actor-critic, đặc biệt là DDPG và PPO.
- Nắm được cách xây dựng và huấn luyện Agent cho cả môi trường điều khiển liên tục và môi trường điều khiển rời rạc.
- Biết cách thiết kế các thành phần cốt lõi của một thuật toán RL, đặc biệt là DDPG, TD3 và PPO.
- Vận dụng được DDPG và PPO, đặc biệt là PPO vào bài toán Edge Offloading phức tạp.
- Có khả năng đánh giá và so sánh hiệu năng, xây dựng các tiêu chí và phân tích sự tác động của các hyperparameters đối với chất lượng huấn luyện.

## B. GIỚI THIỆU

### 1. Deep Deterministic Policy Gradient

#### a. Giới hạn của value-based methods

Sự ra đời của **Deep Deterministic Policy Gradient (DDPG)** gắn liền với quá trình mở rộng học tăng cường từ các môi trường đơn giản sang các bài toán điều khiển thực tế có độ phức tạp cao. Để hiểu rõ động lực phát triển của DDPG, cần nhìn lại những hạn chế của các phương pháp học tăng cường trước đó, đặc biệt là các thuật toán value-based methods như Q-Learning và DQN. Phải khẳng định rằng, sự ra đời của DQN vào năm 2013 đã đánh dấu sự thành công trong việc kết hợp mạng nơ-ron sâu với thuật toán học tăng cường cổ điển, tạo nên dấu ấn lớn đối với cộng đồng AI thời điểm bấy giờ. Tuy nhiên, thành công của DQN lại ẩn chứa một giới hạn cơ bản về bản chất. DQN được thiết kế để hoạt động với **không gian hành động rời rạc và nhỏ**. Ví dụ trong game điều khiển nhân vật đi theo 4 hướng: lên, xuống, trái và phải.

Nhưng điều gì sẽ xảy ra nếu **không gian hành động là liên tục và vô hạn**? Hãy tưởng tượng bạn là một kỹ sư robotics, bạn muốn dạy một cánh tay robot nắm một chiếc cốc. Thay vì 4 hành động riêng biệt như: tay trái, tay phải, tay lên, tay xuống, cánh tay robot phải điều khiển các momen xoắn (torque) của mỗi khớp, mà đó là những giá trị liên tục. Mỗi khớp có thể quay với bất kỳ độ lớn nào từ  $-180^\circ$  đến  $+180^\circ$ , hay với bất kỳ vận tốc nào từ  $-10$  rad/s đến  $+10$  rad/s. Hãy một xe tự lái cần điều chỉnh góc lái từ  $-45^\circ$  đến  $+45^\circ$ . Với DQN, bạn không thể nói “ước tính Q-value cho tất cả các hành động liên tục có thể” - đó là một bài toán vô hạn, không thể giải quyết bằng mạng nơ-ron hữu hạn. DQN đơn giản là **không được thiết kế để xử lý các bài toán với hành động liên tục**.

### b. Nỗ lực tìm kiếm giải pháp

Khi gặp phải hạn chế này, cộng đồng học tăng cường đã cố gắng phát triển các giải pháp thuộc họ policy-based methods với **Policy Gradient methods**. Thay vì ước tính giá trị của các hành động, Policy Gradient methods học trực tiếp một policy - một hàm số nhận input là trạng thái và output là xác suất của các hành động, hoặc trong trường hợp liên tục, output là các tham số của một phân phối hành động. Điều này rất hợp lý cho hành động liên tục. Nhưng trong thực tế, Policy Gradient methods truyền thống như **REINFORCE** hay **Actor-Critic** cơ bản gặp phải các vấn đề nghiêm trọng:

#### \*Độ ổn định kém:

Một trong những khó khăn lớn nhất của các Policy Gradient methods truyền thống là quá trình học thường không ổn định. Trong nhiều trường hợp, Agent có thể học khá tốt trong một khoảng thời gian ngắn nhưng sau đó hiệu năng lại giảm mạnh hoặc thậm chí sụp đổ hoàn toàn. Nguyên nhân cốt lõi nằm ở cách policy được cập nhật trực tiếp thông qua gradient. Trong các thuật toán như REINFORCE, mục tiêu của Agent là tối đa hóa phần thưởng kỳ vọng (expected returns) dài hạn bằng cách điều chỉnh tham số của policy. Về mặt

ý tưởng thì hoàn toàn hợp lý nhưng trong thực tế, reward thường có tính ngẫu nhiên cao và phụ thuộc vào toàn bộ trajectory. Điều này khiến gradient có variance rất lớn. Chỉ cần một số episode có reward bất thường, policy có thể bị cập nhật theo hướng hoàn toàn sai lệch.

Ví dụ, hãy tưởng tượng một robot học cách giữ thăng bằng. Trong vài nghìn lần thử đầu tiên, robot dần học được cách điều chỉnh bánh xe để không bị ngã quá nhanh. Tuy nhiên, chỉ cần một vài episode “may mắn” nơi robot giữ được thăng bằng lâu hơn do yếu tố ngẫu nhiên, policy có thể bị điều chỉnh quá mạnh về một hướng không thực sự tối ưu. Kết quả là sau vài lần update tiếp theo, robot lại mất hoàn toàn khả năng giữ cân bằng.

Đây là một vấn đề cực kỳ nghiêm trọng vì mạng nơ-ron vốn đã nhạy cảm với dữ liệu nhiễu và gradient không ổn định. Ngoài ra, trong Policy Gradient methods truyền thống, policy mới thường thay đổi trực tiếp từ policy cũ mà không có cơ chế giới hạn mức độ thay đổi. Điều này dẫn tới việc Agent có thể “quên” hoàn toàn những hành vi tốt đã học trước đó chỉ sau một số lần cập nhật. Nói cách khác, quá trình học thiếu tính bảo toàn và rất dễ dao động.

### **\*Hiệu quả mẫu thấp:**

Một thuật toán được xem là có sample efficiency cao nếu nó có thể học được policy tốt chỉ với một lượng nhỏ interaction. Ngược lại, nếu Agent cần hàng triệu lần thử nghiệm mới cải thiện được hành vi thì thuật toán đó được xem là có hiệu quả mẫu thấp. Các thuật toán như REINFORCE thường thuộc nhóm on-policy methods. Điều này có nghĩa là dữ liệu chỉ có giá trị đối với policy hiện tại. Sau khi policy được cập nhật, các trajectory cũ gần như không còn được sử dụng nữa vì chúng không còn phản ánh đúng hành vi của policy mới. Kết quả là Agent phải liên tục thu thập dữ liệu mới từ môi trường để tiếp tục học.

Trong simulation, vấn đề này chủ yếu làm tăng thời gian training. Nhưng trong môi trường thực tế, hiệu quả mẫu thấp trở thành một rào cản rất nghiêm trọng. Ví dụ: một drone

thật không thể crash hàng trăm nghìn lần chỉ để học cách bay ổn định, robot công nghiệp không thể liên tục thử sai trong nhiều tuần, hệ thống mạng thật không thể chấp nhận quá nhiều cấu hình lỗi chỉ để Agent khám phá. Mỗi interaction trong môi trường thực đều đi kèm: chi phí thời gian, năng lượng, tài nguyên phần cứng và đôi khi là rủi ro hư hỏng thiết bị. Đây chính là lý do vì sao sample efficiency trở thành tiêu chí cực kỳ quan trọng trong học tăng cường sâu hiện đại.

### \*Hội tụ chậm:

Trong học tăng cường, hội tụ được hiểu là quá trình Agent dần ổn định policy và tiến tới hành vi tối ưu hoặc gần tối ưu. Nguyên nhân đầu tiên cho sự chậm chạp này nằm ở bản chất expected returns. Trong nhiều bài toán, reward không xuất hiện ngay sau mỗi action mà chỉ xuất hiện sau một chuỗi hành động dài. Điều này khiến Agent rất khó xác định hành động nào thực sự đóng góp vào thành công hoặc thất bại. Ngoài ra, vấn đề gradient variance cao trên Policy Gradient methods cũng ảnh hưởng đến quá trình hội tụ. Một bài toán đơn giản có thể cần hàng triệu steps training mới có kết quả khả tín và cải thiện đáng kể hành vi của mình.

### c. Sự ra đời của DDPG

Với các động lực phát triển của Q-learning, DQN, REINFORCE cho đến Actor-Critic architectures, DDPG là điểm giao thoa giữa hai hướng tiếp cận lớn trong học tăng cường: value-based methods và policy-based methods. Ý tưởng đầu tiên xuất phát từ việc thay vì sử dụng stochastic policy, tức là policy sinh ra phân phối xác suất của hành động. Cách tiếp cận này hoạt động tốt trong nhiều trường hợp nhưng lại tạo ra variance lớn khi action space có số chiều cao. Từ đây, Deterministic Policy Gradient (DPG) đề xuất thay vì sinh phân phối xác suất, policy sinh trực tiếp action xác định:

$$a = \mu(s)$$

Ý tưởng deterministic policy đặc biệt phù hợp với bài toán continuous control vì: giảm variance của gradient, đơn giản hóa optimization và hiệu quả hơn trong không gian action liên tục nhiều chiều. Tuy nhiên, DPG nguyên thủy vẫn gặp khó khăn khi áp dụng vào các môi trường lớn và phức tạp vì nó chưa tận dụng được sức mạnh của Deep Learning.

Sự phát triển mạnh mẽ của Deep Neural Network trong giai đoạn 2013-2015 đã tạo tiền đề cho việc mở rộng DPG bằng học sâu. Năm 2015, Timothy Lillicrap và nhóm nghiên cứu tại DeepMind đã giới thiệu DDPG. Thuật toán này có thể xem là sự kết hợp giữa:

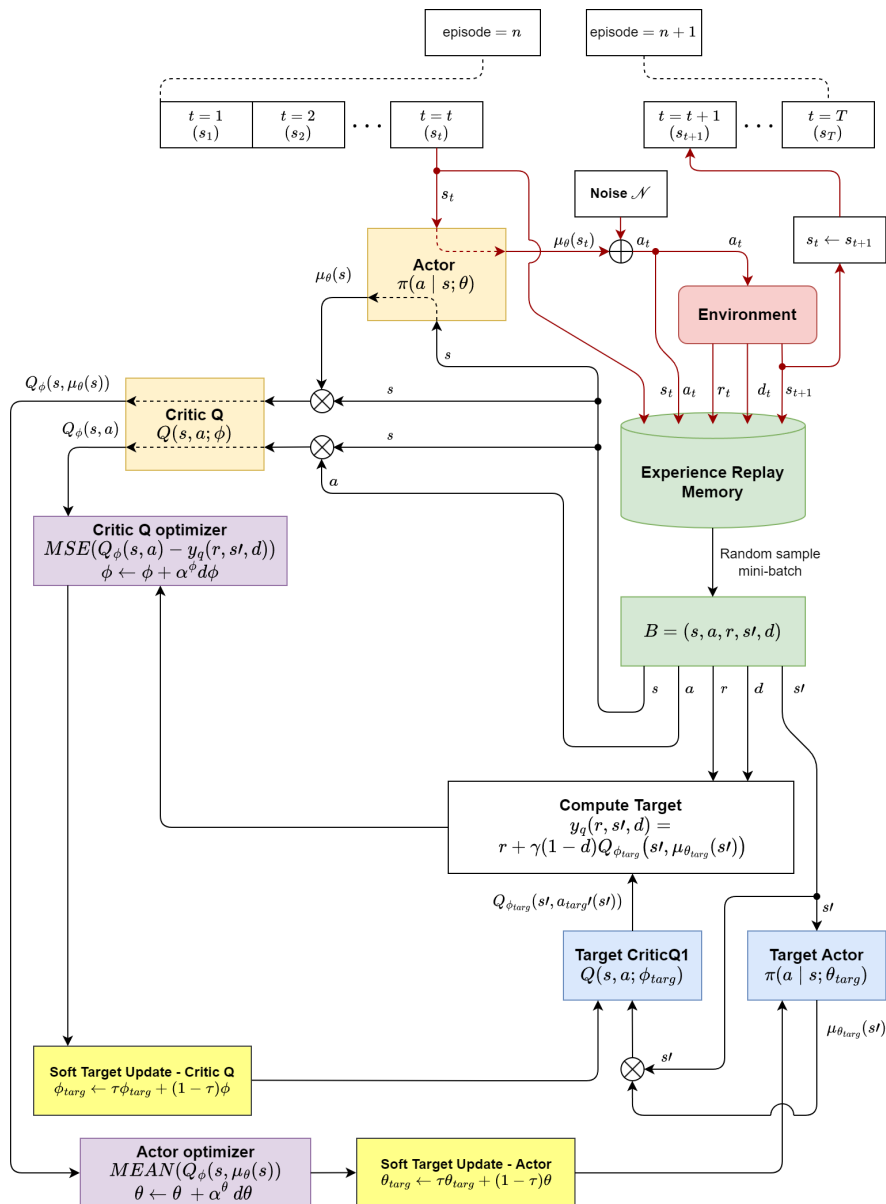
- Deterministic Policy Gradient,
- Actor-Critic architecture,
- và các kỹ thuật ổn định hóa của DQN: replay buffer và target networks.

DDPG sử dụng một kỹ thuật gọi là target networks - giữ các bản sao cũ của Q-function và policy, cập nhật chúng rất từ từ. Bằng cách này, target được cố định trong một thời gian, cho phép learning ổn định hơn. Ngoài ra, do DDPG sử dụng deterministic policy nên exploration không còn được thực hiện bằng sampling từ phân phối xác suất như trong stochastic policy. Thay vào đó, thuật toán thêm noise trực tiếp vào action:

$$a_t = \mu(s_t) + N$$

Ví dụ: robot dự định quay bánh xe với lực 0.4, exploration noise có thể khiến nó thử 0.35 hoặc 0.45. Điều này giúp Agent tiếp tục khám phá môi trường thay vì luôn lặp lại cùng một hành động. Sự ra đời của DDPG có ý nghĩa rất lớn vì lần đầu tiên học tăng cường sâu có thể xử lý continuous control một cách tương đối hiệu quả. Thuật toán nhanh chóng được áp dụng vào: robotic arm control, drone navigation, autonomous vehicle simulation, energy optimization, network resource management và nhiều bài toán control systems khác.

Tuy nhiên, mặc dù rất tiên tiến ở thời điểm ra đời, DDPG vẫn tồn tại nhiều hạn chế. Vấn đề lớn nhất là training instability. DDPG rất nhạy với: hyperparameters, exploration noise, reward scaling, learning rate. Trong nhiều trường hợp: critic đánh giá Q-value quá cao, actor học sai hướng, toàn bộ policy bị collapse hoặc exploration chưa đủ mạnh trong các môi trường phức tạp.



Hình 1. Quá trình hoạt động của DDPG<sup>1</sup>

<sup>1</sup> <https://wikidocs.net/204479>



Những hạn chế này dẫn tới sự phát triển của hàng loạt thuật toán kế thừa nhằm cải thiện DDPG như: Twin Delayed DDPG (TD3), Soft Actor-Critic (SAC) và nhiều thuật toán actor-critic tiên tiến khác.

## 2. Proximal Policy Optimization

Proximal Policy Optimization (PPO)<sup>2</sup> là một trong những thuật toán học tăng cường hiện đại nổi bật và được sử dụng rộng rãi nhất hiện nay. Thuật toán này được giới thiệu bởi OpenAI vào năm 2017 nhằm giải quyết một trong những vấn đề lớn nhất của học tăng cường sâu: làm thế nào để cập nhật policy đủ mạnh - để học hiệu quả - nhưng vẫn đủ ổn định để tránh làm sụp đổ quá trình huấn luyện.

PPO thuộc nhóm:

- policy-based methods,
- đồng thời sử dụng kiến trúc Actor-Critic.

Thuật toán này được xem là sự kế thừa và đơn giản hóa từ Trust Region Policy Optimization (TRPO), một phương pháp tối ưu policy rất mạnh nhưng phức tạp trong triển khai thực tế. PPO giữ lại tư tưởng cốt lõi của TRPO - tức là giới hạn mức độ thay đổi của policy sau mỗi lần cập nhật - nhưng sử dụng cơ chế đơn giản và hiệu quả hơn nhiều.

### a. Bối cảnh phát triển

Để hiểu vì sao PPO ra đời, cần nhìn lại sự phát triển của học tăng cường trước năm 2017. Chứng kiến với sự thành công của các thuật toán value-based methods đại diện như DQN, đồng thời là các nghiên cứu về khả năng ứng dụng đáng kể của các policy-based methods. Tuy nhiên, các Policy Gradient truyền thống lại gặp nhiều vấn đề nghiêm trọng. Một trong những khó khăn lớn nhất ban đầu là **quá trình học rất thiếu ổn định**. Nguyên nhân nằm ở

---

<sup>2</sup> Schulman, John et al. "Proximal Policy Optimization Algorithms." ArXiv abs/1707.06347 (2017).



chỗ policy mới có thể quá khác policy cũ, khiến agent “quên” toàn bộ hành vi tốt đã học trước đó. Nói cách khác, các thuật toán Policy Gradient truyền thống thiếu một cơ chế kiểm soát mức độ thay đổi của policy sau mỗi lần optimization.

Để giải quyết vấn đề này, OpenAI phát triển thuật toán **Trust Region Policy Optimization (TRPO)**<sup>3</sup>. Ý tưởng cốt lõi của TRPO là policy không nên thay đổi quá xa khỏi policy cũ sau mỗi lần update, hay có thể hiểu rằng optimization phải được thực hiện trong một “vùng an toàn” gọi là trust region. Kỹ thuật được sử dụng để xem khoảng cách và cập nhật thường được đo bằng **KL-Divergence**. Về mặt ý tưởng mặc dù rất hiệu quả nhưng TRPO lại rất khó triển khai trong thực tế do tính phức tạp của nó. Đây chính là bối cảnh trực tiếp dẫn tới sự ra đời của PPO.

### b. Ý tưởng cốt lõi của PPO

PPO được phát triển với mục tiêu rất rõ ràng: giữ lại độ ổn định của TRPO, nhưng đơn giản hóa optimization để dễ triển khai hơn. Toàn bộ quá trình học của Agent được xây dựng xoay quanh một mục tiêu rất quan trọng: cải thiện policy nhưng vẫn giữ cho quá trình cập nhật đủ ổn định để tránh làm sụp đổ những gì Agent đã học trước đó. Đây cũng là điểm khác biệt và tạo nên “thương hiệu” cho PPO.

Ý tưởng trung tâm của PPO thường được mô tả bằng khái niệm “proximal”, tức là gần kề. Thay vì cho phép policy thay đổi tự do sau mỗi đợt cập nhật, PPO cố tình giới hạn mức độ thay đổi của policy mới so với policy cũ. Cách làm nổi tiếng nhất của PPO là dùng một hàm mục tiêu bị chặn bởi clipping. Cụ thể, ta xét tỉ lệ xác suất giữa policy mới và policy cũ tại cùng một hành động đã được lấy mẫu:

<sup>3</sup> Schulman, J., Levine, S., Abbeel, P., Jordan, M., Moritz, P. (2015). Trust Region Policy Optimization. Proceedings of the 32nd International Conference on Machine Learning in Proceedings of Machine Learning Research, pp. 37:1889-1897

$$r_t(\theta) = \frac{\pi(a_t | s_t, \theta)}{\pi(a_t | s_t, \theta_{old})}$$

Về trượt giá:

- Nếu  $r_t(\theta) > 1$  rất nhiều  $\rightarrow$  nghĩa là policy mới đang tăng xác suất của hành động đó quá mạnh;
- Nếu  $r_t(\theta) < 1$  quá nhiều  $\rightarrow$  policy mới đang giảm xác suất của hành động đó quá mạnh.

PPO đặt ra một biên an toàn bằng cách “cắt” tỉ lệ này trong một khoảng gần 1, thường là  $[1 - \epsilon, 1 + \epsilon]$ . Khi đó, hàm mục tiêu clipped surrogate objective có dạng:

$$L(\theta) = \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta); 1 - \epsilon, 1 + \epsilon)A_t)]$$

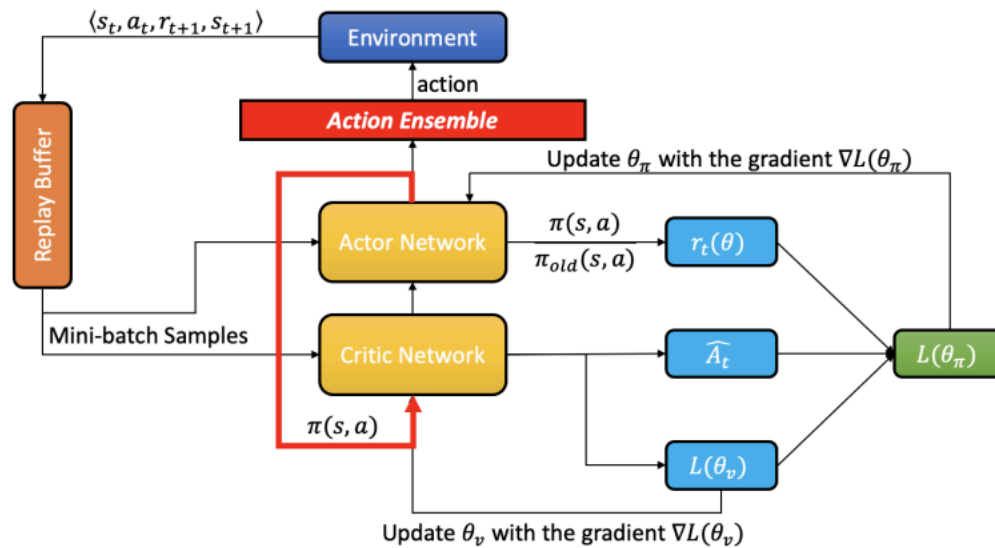
Ở đây,  $A_t$  là **advantage** estimate, tức là mức độ tốt của hành động tại thời điểm  $t$  so với kỳ vọng trung bình. Công thức này là trái tim kỹ thuật của PPO. Ý tưởng của nó không quá khó hiểu:

- Nếu một hành động có advantage dương  $\rightarrow$  policy nên tăng xác suất của nó, nhưng chỉ đến một mức vừa phải;
- Nếu advantage âm  $\rightarrow$  policy nên giảm xác suất, cũng chỉ trong giới hạn an toàn.

Khi policy cố tình đi quá xa, phần “min” trong công thức sẽ làm cho lợi ích cập nhật đó bị chặn lại. Nhờ vậy, quá trình học trở nên ổn định hơn nhiều so với các Policy Gradient thuần túy. Điểm đáng chú ý là PPO không tìm cách tối ưu theo kiểu “tham lam tuyệt đối”. Nó chấp nhận tiến bộ chậm hơn một chút để đổi lấy độ ổn định cao hơn. Đây là một quyết định rất quan trọng về mặt thiết kế thuật toán. Từ đó, đạt được tính đơn giản, độ ổn định và hiệu quả thực nghiệm cao.

### c. Xây dựng và hiện thực PPO

Sau khi hiểu được ý tưởng cốt lõi của PPO, bước tiếp theo là xem xét cách thuật toán này được xây dựng và triển khai trong thực tế. Điều quan trọng cần hiểu là PPO không chỉ là một công thức toán học đơn lẻ, mà là một quy trình học hoàn chỉnh bao gồm: thu thập dữ liệu từ môi trường → đánh giá chất lượng hành động → tối ưu policy → ổn định quá trình cập nhật, → lặp lại liên tục cho đến khi Agent học được hành vi mong muốn.



Hình 2. Quá trình hoạt động của PPO<sup>4</sup>

- 1) Trong hầu hết các triển khai hiện đại, PPO sử dụng kiến trúc Actor-Critic và bắt đầu bằng việc khởi tạo policy network và value network dựa trên Deep Neural Network.
- 2) Sau đó Agent chạy trong môi trường để thu thập một batch trajectory. Tập dữ liệu này thường gồm trạng thái, hành động, reward, log probability của hành động dưới policy cũ, và value estimate của Critic. Đây là bước quan trọng vì PPO là on-policy: dữ liệu phải phản ánh policy hiện tại hoặc rất gần hiện tại.
- 3) Sau khi thu thập đủ dữ liệu, thuật toán bắt đầu giai đoạn đánh giá chất lượng hành động. Đây là nơi Advantage Function đóng vai trò trung tâm. Trong thực tế, có khá

<sup>4</sup> Wang, Yongliang and Hamidreza Kasaei. "Obstacle Avoidance for Robotic Manipulator in Joint Space via Improved Proximal Policy Optimization." ArXiv abs/2210.00803 (2022).

nhiều cách hiện thực advantage estimate, tùy theo bài toán thì nên chọn cách hiện thực sao cho cân bằng giữa độ lệch, phương sai và chi phí tính toán. Trong các kỹ thuật hiện đại, PPO thường sử dụng kỹ thuật **Generalized Advantage Estimation (GAE)** để tính advantage.

- 4) Tiếp đến là tính tỷ lệ  $rt(\theta)$  để xác định sự thay đổi xác suất thực hiện action so với trước khi cập nhật. Đồng thời sử dụng cơ chế clipping để giới hạn mức độ thay đổi của policy trong một khoảng an toàn.
- 5) Và cuối cùng là Objective của PPO được tổng hợp lại dựa trên các thông số trên và cập nhật lại trong số. Đồng thời loss của value network cũng sẽ được cập nhật.

---

**Algorithm 1** on-policy PPO
 

---

Initialize network parameters:  $\theta$  for policy network,  $\phi$  for value network, max training iterations  $M$  and training batch  $D$ .

```

1: for training iterations = 1 to  $M$  do
2:   Clear the training batch  $D$ .
3:   for each RL collect step  $t$  do
4:     Observe the environment state  $o_t$ . Select action  $a_t$ 
       according to the current policy  $\pi_\theta(a_t|o_t)$ . Execute
       action  $a_t$ . Obtain the reward  $r_t$ . Then environment
       transitions to the next state  $o_{t+1}$ .
5:   end for
6:   Compute advantage estimate  $\hat{A}$  via GAE and add the
       experiences  $(o_t, a_t, r_t, o_{t+1})$  to training batch  $B$ .
7:   for each RL training step do
8:     Recompute advantage estimate  $\hat{A}$  via GAE.
9:     Split the training batch  $D$  into  $K$  mini-batches  $\mathcal{B}$ 
       according to the batch_size.
10:    for mini-batch  $k = 1$  to  $K$  do
11:      Compute the  $ppo\_total\_loss$ .
12:      Update critic and actor network w.r.t the
        $ppo\_total\_loss$ .
13:    end for
14:  end for
15: end for
16: return  $V_\phi, \pi_\theta$ 
  
```

---

Hình 3. Pseudocode PPO<sup>5</sup>

<sup>5</sup> [https://di-engine-docs.readthedocs.io/en/latest/12\\_policies/ppo.html](https://di-engine-docs.readthedocs.io/en/latest/12_policies/ppo.html)

Một vòng PPO thường không chỉ tối ưu một lần trên batch dữ liệu thu được, mà lặp lại nhiều epoch trên mini-batch nhỏ. Đây là điểm kỹ thuật rất đáng chú ý. Mặc dù PPO là on-policy, nó vẫn tái sử dụng batch rollout trong một số epoch ngắn để tận dụng dữ liệu tốt hơn. Một ý tưởng khác cũng thường được ứng dụng trong PPO được gọi là **Entropy Bonus**. Trong PPO, policy được tối ưu để tối đa hóa reward. Tuy nhiên, nếu chỉ tập trung vào reward, policy thường nhanh chóng trở nên “tham lam” (greedy). Agent sẽ liên tục khai thác những action hiện tại có vẻ tốt mà giảm dần exploration. Entropy bonus giúp PPO duy trì khả năng exploration trong suốt quá trình training bằng cách đo mức độ ngẫu nhiên hoặc độ phân tán của một phân phối xác suất. Cuối cùng, cập nhật lại Objective bao gồm cả policy loss, value loss và entropy bonus.

## C. THỰC HÀNH

### 1. Thực hiện hiện thực thuật toán DDPG trên môi trường MountainCarContinuous-v0

**Mô tả:** Sinh viên tìm hiểu về các ý tưởng và hiện thực thuật toán DDPG trên môi trường MountainCarContinuous-v0 có sẵn trên Gymnasium.

- **Bước 1:** Nhìn nhận bài toán.

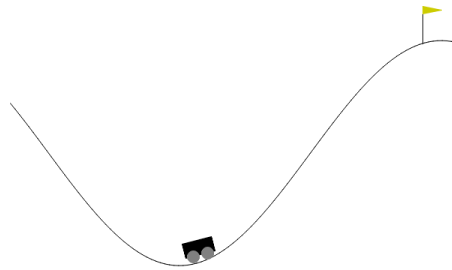
[MountainCar Continuous](#) là một môi trường học tăng cường thuộc thư viện Gymnasium, thường được sử dụng để nghiên cứu và thực hành các thuật toán điều khiển liên tục như DDPG hay TD3.... Trong môi trường này, Agent điều khiển một chiếc xe nhỏ nằm giữa hai ngọn đồi và nhiệm vụ của nó là đưa xe lên tới đỉnh đồi bên phải. Tuy nhiên, động cơ của xe không đủ mạnh để leo thẳng lên đỉnh, do đó Agent phải học cách di chuyển qua lại để tạo đà trước khi có thể đạt được mục tiêu.

Khác với phiên bản MountainCar truyền thống sử dụng discrete action space, MountainCar Continuous sử dụng continuous action space, nghĩa là Agent phải sinh ra một giá trị lực liên tục trong khoảng:  $a \in [-1, 1]$

Trong đó:

- giá trị âm biểu diễn lực đẩy sang trái,
- giá trị dương biểu diễn lực đẩy sang phải,
- và độ lớn thể hiện cường độ tăng tốc.

Điều này khiến môi trường trở thành một bài toán điều khiển liên tục điển hình, rất phù hợp để minh họa cho ưu điểm của các thuật toán actor-critic như DDPG. Agent không chỉ phải quyết định hướng di chuyển mà còn phải học cách điều chỉnh lực đẩy một cách chính xác để tối ưu khả năng tạo động lượng và tiết kiệm năng lượng.



- **Bước 2:** Tìm hiểu về các biểu diễn Markov Decision Process (MDP) của bài toán.

- ? *Observation Space* mô tả điều gì?
- ? *Action Space* mô tả điều gì?
- ? *Reward* được định nghĩa như thế nào?
- ? Nguyên tắc hoạt động của Agent?
- ? Khi nào thì bài toán kết thúc?

- **Bước 3:** Cài đặt thư viện liên quan: `pip install -r requirements.txt`

- **Bước 4:** Mở file `Lab5.1-st.ipynb`, thực hiện chạy và hoàn thành *Part 1* đã được cung cấp sẵn.

Thực hiện tạo `ReplayBuffer` để có khả năng tái sử dụng trải nghiệm, trong đó định nghĩa hàm `sample` để tạo batch dữ liệu một cách ngẫu nhiên.

```
def sample(self, batch_size=64):
    # Randomly sample a batch of transitions from the replay buffer and convert them to
    # PyTorch tensors
    sampled_indices = np.random.choice(len(self.transition_buffer), batch_size,
                                       replace=False)
    # Retrieve the sampled transitions and unzip them into separate arrays for states,
    # actions, rewards, next states, and done flags
    sampled_batch = [self.transition_buffer[index_value] for index_value in
                     sampled_indices]
    states_array, actions_array, rewards_array, next_states_array, dones_array =
    map(np.array, zip(*sampled_batch))
    return (
        torch.tensor(states_array, dtype=torch.float32, device=device),
        torch.tensor(actions_array, dtype=torch.float32, device=device),
        torch.tensor(rewards_array, dtype=torch.float32, device=device).unsqueeze(1),
        torch.tensor(next_states_array, dtype=torch.float32, device=device),
        torch.tensor(dones_array, dtype=torch.float32, device=device).unsqueeze(1),
    )
```

? Tại sao phải lấy dữ liệu ra một cách ngẫu nhiên?

Cần khởi tạo **4 mạng nơ-ron** lần lượt: Online Actor, Target Actor, Online Critic, Target Critic.

```
# Initialize the online and target networks for both the actor and critic
ddpg_online_actor = DDPGActor(ddpg_state_dimension, ddp_g_action_dimension,
                              ddp_g_max_action_value).to(device)
ddpg_target_actor = DDPGActor(ddpg_state_dimension, ddp_g_action_dimension,
                              ddp_g_max_action_value).to(device)
ddpg_online_critic = DDPGCritic(ddpg_state_dimension, ddp_g_action_dimension).to(device)
ddpg_target_critic = DDPGCritic(ddpg_state_dimension, ddp_g_action_dimension).to(device)
```

Sau khi đã thu thập đủ trải nghiệm trong quá trình tương tác, cần tìm các loss cho các Actor, Critic network để cập nhật trọng số.

```
sampled_states, sampled_actions, sampled_rewards, sampled_next_states, sampled_dones =
ddpg_replay_buffer.sample(self.batch_size)

with torch.no_grad():
    target_actions = ddp_g_target_actor(sampled_next_states)
    target_q_values = ddp_g_target_critic(sampled_next_states, target_actions)
```



```

        td_target_values = sampled_rewards + self.discount_gamma * (1.0 - sampled_dones) *
target_q_values

        current_q_values = ddpq_online_critic(sampled_states, sampled_actions)
        critic_loss_value = nn.MSELoss()(current_q_values, td_target_values)
        ...

        actor_loss_value = -ddpg_online_critic(sampled_states,
ddpg_online_actor(sampled_states)).mean()
        ...

```

## 2. Hiện thực thuật toán PPO trên môi trường CartPole-v1

**Mô tả:** Sinh viên thực hiện hiện thực thuật toán PPO trên môi trường CartPole-v1, thực hiện tìm hiểu về các thông số, cấu hình của PPO trong quá trình triển khai.

- **Bước 1:** Tiếp tục với file **Lab5.1-st.ipynb**, thực hiện chạy và hoàn thiện *Part 3* đã được cung cấp sẵn.
- **Bước 2:** Cách tạo các mini-batches để thực hiện tìm loss cho policy network sau khi đã thu thập được các trải nghiệm thông qua việc tương tác với môi trường.

```

for _ in range(epochs_count):
    # Shuffle the indices of the collected samples to ensure that each optimization
epoch processes the data in a different order,
    # which can help improve generalization and prevent overfitting to specific
sequences of transitions.
    np.random.shuffle(index_array)
    for start_index in range(0, sample_count, minibatch_size):
        # Create mini-batches of data for the current optimization step by selecting a
subset of the shuffled indices
        # and retrieving the corresponding states, actions, log probabilities,
advantages, and returns from the tensors.
        batch_indices = index_array[start_index:start_index + minibatch_size]
        batch_states = state_tensor[batch_indices]
        batch_actions = action_tensor[batch_indices]
        batch_old_log_probabilities = old_log_probability_tensor[batch_indices]
        batch_advantages = advantage_tensor[batch_indices]
        batch_returns = return_tensor[batch_indices]

```

- **Bước 3:** Thực hiện chạy huấn luyện và tiến hành đánh giá mức độ hiệu quả PPO trên môi trường **CartPole-v1**

**?** Bạn có nhận xét gì về kết quả đạt được? Đồng thời, bạn có nhận xét gì về độ ổn định qua quá trình training?

## D. YÊU CẦU & NỘI BÀI

### 1. Yêu cầu

1. Đối với *MountainCar Continuous Environment*, dựa trên các đoạn mã được cung cấp, hoàn thành các block code còn lại để tích hợp thuật toán DDPG và TD3 vào môi trường CartPole tại file **Lab5.1-st.ipynb, Phần 1,2**. Thực hiện cấu hình tùy chỉnh số episodes, vẽ reward curve của 2 thuật toán trên và tiến hành phân tích, so sánh về tốc độ học tập và khả năng hội tụ như thế nào đối với cấu hình hiện tại. Tìm hiểu trên mã nguồn Stable-Baseline3 và các nguồn tài liệu khác để trình bày về cải tiến của TD3 so với DDPG.

**Lưu ý:** Chạy lại với nhiều seed khác nhau và tổng hợp kết quả ( $\geq 5$ ).

2. Thực hiện thay đổi thông số **tau** và **exploration noise** theo các kịch bản:

- $\tau \in \{0.005, 0.05, 1.0\}$
- $\text{exploration noise} \in \{0.01, 0.15, 0.3, 0.5\}$

Thực hiện huấn luyện từng kịch bản trên từng thông số và báo cáo về: Độ ổn định và mức độ hội tụ đối với thông số  $\tau$ ; Khả năng exploration đối với thông số exploration noise.

3. Thực hiện điều chỉnh sao cho DDPG với 100 test steps thì success rate  $> 80\%$ . Giải thích cho sự thay đổi để đạt được mục tiêu, phân tích riêng lẻ các phần và tổng hợp.

**Gợi ý:** Hãy thử với kiến trúc mạng nơ-ron,  $\tau$ , batch\_size, exploration noise, learning rates, ddp\_g\_episode\_count, buffer\_capacity và số lượng training episodes.

4. Đối với *CartPole Environment*, dựa trên các đoạn mã được cung cấp, hoàn thành các block code còn lại để tích hợp thuật toán PPO vào môi trường CartPole tại file **Lab5.1-**

st.ipynb, Phần 3.4. Thực hiện thay đổi thông số **Clip Ratio** và **GAE Lambda** theo các kịch bản:

- clip ratio  $\in \{0.05, 0.2, 0.5\}$
- gae lambda  $\in \{0.5, 0.95, 0.99\}$

Thực hiện huấn luyện từng kịch bản trên từng thông số và báo cáo về: tốc độ học tập và khả năng cân bằng đối với thông số clip ratio ; Mức độ variance và bias đối với thông số gae lambda.

5. Đối với *Edge Offloading Environment*, thực hiện tích hợp thuật toán PPO đã xây dựng ở các nội dung trước và kết hợp với kết quả của A2C tại bài thực hành số 4 vào môi trường Edge Offloading đã xây dựng. Thực hiện đánh giá về độ ổn định của 2 thuật toán này trong quá trình huấn luyện thông qua reward curve, avg return, std return, max return,... hoặc các biểu đồ, thông số khác.

6. Trên các policy đã xây dựng: Random, Device-only, Edge-only và Greedy latency. Vẽ bảng và tiến hành so sánh, đánh giá các thông số: Avg Latency, P95 Latency, Avg Reward, Device % / Edge % và Gap to Greedy. Bạn có nhận xét gì khi so sánh giá giữa PPO và các policy này.

**Lưu ý:** Có thể điều chỉnh các hyperparameters của PPO để xem khả năng có thể biểu diễn được gần với policy Greedy latency.

## 2. Yêu cầu nộp bài

- Sinh viên tìm hiểu và thực hành theo hướng dẫn. Thực hiện **theo nhóm**.
- Sinh viên báo cáo kết quả thực hiện và nộp bài bằng file. Trong đó:
  - Trình bày chi tiết quá trình thực hành và trả lời các câu hỏi nếu có (kèm theo các ảnh chụp màn hình tương ứng).
  - Giải thích các kết quả đạt được.
  - Tải mẫu báo cáo thực hành và trình bày theo mẫu được cung cấp.

Nén tất cả các file và đặt tên file theo định dạng theo mẫu:

**NhomY-LabX\_MSSV1\_MSSV2**

Ví dụ: Nhom1-Lab05\_29520001\_29520002\_29520003

- Nộp báo cáo trên theo thời gian đã thống nhất tại website môn học.
- Các bài nộp không tuân theo yêu cầu sẽ **KHÔNG** được chấm điểm.

**HẾT**

Chúc các bạn hoàn thành tốt bài thực hành!